



دانشگاه صنعتی اصفهان

دانشکده مهندسی کامپیوتر

مدیریت و بهبود هوشمند کیفیت خدمت (QoS) در شبکه‌های SDN مبتنی بر یادگیری ماشین با رویکرد صف‌بندی پویا و مسیریابی چندگانه

گزارش نهایی پروژه تحقیقاتی / کاربردی

دانشجو : شایان رواخواه

شماره دانشجویی : 40123703

استاد راهنما : دکتر مسعود رضا هاشمی

مقطع : کارشناسی ارشد

تابستان ۱۴۰۴

فهرست مطالب

۳	فصل اول: مقدمه و کلیات تحقیق
۳	۱-۱ مقدمه
۳	۱-۲ بیان مسئله
۴	۱-۳ اهداف و دستاوردهای پروژه
۵	۱-۴ ساختار گزارش
۶	فصل دوم: مفاهیم پایه و مرور منابع
۶	۲-۱ شبکه‌های مبتنی بر نرم‌افزار (SDN)
۷	۲-۲ محیط شبیه‌سازی و کنترلر POX
۷	۲-۳ کیفیت خدمت (QoS) و تکنیک‌های مدیریت ترافیک
۸	۲-۴ یادگیری ماشین در مدیریت هوشمند شبکه
۹	فصل سوم: معماری سیستم و پیاده‌سازی فنی
۹	۳-۱ نگاه کلی به معماری
۹	۳-۲ تشریح توپولوژی‌های شبکه
۱۱	۳-۳ ساختار ۳ کنترلر POX
۱۳	۳-۴ پایپ‌لاین یادگیری ماشین

۱۳ ۳-۵ چرخه کاری سیستم

۱۴ فصل چهارم: ارزیابی، سناریوهای تست و تحلیل نتایج

۱۴ ۴-۱ تا ۳-۴ مقدمه، پیش‌نیازها و تولید دیتاست

۱۵ ۴-۴ سناریوی تست اول: صف‌بندی HTB

۱۷ ۴-۵ سناریوی تست دوم: مسیریابی چندگانه

۱۹ ۴-۶ جدول مقایسه جامع

۲۰ فصل پنجم: نتیجه‌گیری و پیشنهادات

فصل اول: مقدمه و کلیات تحقیق

۱-۱ مقدمه

با گسترش برنامه‌های کاربردی حساس به تاخیر مانند بازی‌های آنلاین و پخش زنده ویدیو، مدیریت منابع شبکه و تضمین کیفیت خدمت (QoS) به یکی از چالش‌های اصلی تبدیل شده است. شبکه‌های سنتی به دلیل پیکربندی ایستا، توانایی واکنش سریع به ازدحام‌های ناگهانی را ندارند. شبکه‌های مبتنی بر نرم‌افزار (SDN) با جداسازی لایه کنترل از لایه داده، امکان تصمیم‌گیری متمرکز را فراهم می‌کنند. ترکیب این معماری با الگوریتم‌های یادگیری ماشین، بستری هوشمند برای تحلیل زنده ترافیک و اعمال سیاست‌های دینامیک ایجاد می‌کند.

۱-۲ بیان مسئله

ورود ناگهانی ترافیک‌های سنگین مانند دانلودهای حجیم، باعث پر شدن ظرفیت لینک‌ها، افزایش تاخیر و افت بسته‌ها برای ترافیک‌های حساس به زمان (مانند ترافیک UDP مربوط به Gaming) می‌شود. برای حل این مشکل، تکنیک‌های مختلفی مانند مدیریت صف‌بندی (Queuing) و مسیریابی چندگانه (Multi-Path Rerouting) وجود دارد. اما اجرای ایستای این روش‌ها ناکارآمد است؛ بنابراین نیاز به سیستمی هوشمند است که ویژگی‌های آماری شبکه را به صورت زنده استخراج کند، با کمک مدل یادگیری ماشین آموزش‌دیده، وقوع ازدحام را پیش‌بینی و تشخیص دهد و متناسب با توپولوژی شبکه، سیاست QoS مربوطه را اعمال کند.

چالش اصلی: چگونه می‌توان در کمتر از چند ثانیه، ازدحام ناگهانی شبکه را تشخیص داد و به صورت خودکار ترافیک حساس را بدون قطعی حفظ کرد؟

۱-۳ اهداف و دستاوردهای پروژه

هدف اصلی این پروژه، طراحی و پیاده‌سازی یک سیستم مدیریت خودکار QoS در محیط شبیه‌سازی SDN با دو رویکرد مبتنی بر یادگیری ماشین است. دستاوردهای کلیدی این تحقیق عبارتند از:

طراحی و پیاده‌سازی ۲ توپولوژی شبکه در مینی‌نت:

- **توپولوژی اول (مبتنی بر صف‌بندی):** ساختاری که در آن مدیریت کیفیت خدمت از طریق اولویت‌بندی ترافیک و تخصیص صف‌های اختصاصی (Linux HTB Queues) روی لینک‌ها پیاده‌سازی شده است.
- **توپولوژی دوم (مبتنی بر چندمسیره):** ساختار ۸ سوئیچی دارای ۳ مسیر موازی مجزا بین مبدا و مقصد که مدیریت کیفیت خدمت در آن از طریق توزیع بار و هدایت مجدد ترافیک انجام می‌شود.

توسعه ۳ ماژول کنترلر مجزا در بستر POX:

- **کنترلر اول (جمع‌آوری داده):** صرفاً جهت پایش شبکه و استخراج ویژگی‌های آماری برای تولید فایل دیتاست و آموزش مدل یادگیری ماشین.
- **کنترلر دوم (مدیریت صف‌بندی مبتنی بر ML):** کنترلر هوشمند مخصوص توپولوژی اول که با بارگذاری مدل یادگیری ماشین، به محض تشخیص ازدحام، پهنای باند و اولویت صف‌ها را تغییر می‌دهد.
- **کنترلر سوم (مدیریت چندمسیره مبتنی بر ML):** کنترلر هوشمند مخصوص توپولوژی دوم که با تشخیص شرایط ازدحام، ترافیک‌های حساس را به مسیرهای مجزا و خلوت منتقل می‌کند.

تفکیک و هدایت هوشمند ترافیک:

- تضمین اولویت و پهنای باند ترافیک حساس Gaming با استفاده از تشخیص هوشمند مدل ML در هر دو رویکرد
- ایزوله‌سازی ترافیک سنگین دانلود و وب از ترافیک‌های حساس به تاخیر
- کاهش محسوس تاخیر (RTT) و افت بسته در شرایط ازدحام شدید

۱-۴ ساختار گزارش

این گزارش در ۵ فصل تنظیم شده است: فصل اول به کلیات و اهداف می‌پردازد؛ فصل دوم مفاهیم پایه SDN، کنترلر POX و یادگیری ماشین را مرور می‌کند؛ فصل سوم معماری سیستم، ۲ توپولوژی و ۳ کنترلر را تشریح می‌کند؛ فصل چهارم سناریوهای تست و نتایج را ارائه می‌دهد؛ و فصل پنجم شامل نتیجه‌گیری و پیشنهادات آتی است.

فصل دوم: مفاهیم پایه و مرور منابع

۲-۱ شبکه‌های مبتنی بر نرم‌افزار (SDN)

در معماری سنتی شبکه، بخش کنترل (تصمیم‌گیری) و بخش داده (ارسال پکت‌ها) هر دو درون تجهیزات فیزیکی مانند سوئیچ‌ها و روترها قرار دارند. این ساختار باعث می‌شود اعمال تغییرات یا سیاست‌های جدید زمان‌بر و دشوار باشد.

شبکه‌های مبتنی بر نرم‌افزار (SDN) با جداسازی این دو لایه، معماری شبکه را به سه سطح اصلی تقسیم می‌کنند:

لایه کنترل (Control Plane)

یک نرم‌افزار متمرکز به نام کنترلر که دید کاملی نسبت به کل شبکه دارد و جدول جریان (Flow Table) سوئیچ‌ها را مدیریت می‌کند.

لایه زیرساخت (Data Plane)

شامل سوئیچ‌های سخت‌افزاری یا نرم‌افزاری (مانند Open vSwitch) که تنها وظیفه هدایت بسته‌ها بر اساس قوانین دریافتی را بر عهده دارند.

لایه کاربرد (Application Plane)

برنامه‌هایی که روی کنترلر اجرا شده و منطق شبکه مانند مسیریابی، QoS یا امنیت را پیاده‌سازی می‌کنند. ارتباط بین لایه کنترل و لایه داده از طریق پروتکل‌های استاندارد مانند **OpenFlow** برقرار می‌شود. در OpenFlow، کنترلر می‌تواند با ارسال پیام‌هایی مانند **FlowMod** قوانین جدیدی را روی سوئیچ‌ها ثبت کند یا با پیام **StatsRequest** آمار ترافیک را دریافت کند.

۲-۲ محیط شبیه‌سازی و کنترلر POX

برای پیاده‌سازی و تست سناریوهای شبکه بدون نیاز به تجهیزات گران‌قیمت واقعی، از دو ابزار اصلی استفاده شده است:

- **شبیه‌ساز مینی‌نت (Mininet):** ابزاری استاندارد در لینوکس که امکان ایجاد توپولوژی‌های واقعی شبکه شامل میزبان‌ها (Hosts)، سوئیچ‌های مجازی (OVS) و لینک‌های ارتباطی با پارامترهای قابل تنظیم (مانند پهنای باند و تاخیر) را فراهم می‌کند.
- **کنترلر POX:** یک کنترلر OpenFlow نوشته‌شده به زبان پایتون که به دلیل سادگی، انعطاف‌پذیری بالا و ساختار مبتنی بر رویداد (Event-driven)، بستری مناسب برای پیاده‌سازی الگوریتم‌های مدیریت ترافیک و استخراج

آمار زنده شبکه فراهم می‌سازد.

۲-۳ کیفیت خدمت (QoS) و تکنیک‌های مدیریت ترافیک

کیفیت خدمت به مجموعه‌ای از تکنیک‌ها گفته می‌شود که برای تضمین پارامترهای کلیدی شبکه مانند **تاخیر (Latency)**، **نوسان تاخیر (Jitter)**، **نرخ افت بسته (Packet Loss)** و **پهنای باند** استفاده می‌شوند. ترافیک‌های مختلف نیازهای متفاوتی دارند؛ به عنوان مثال ترافیک بازی‌های آنلاین (UDP) به شدت به تاخیر حساس است، در حالی که ترافیک دانلود (TCP) نیازمند پهنای باند بالا است.

در این پروژه از دو رویکرد اصلی برای تامین QoS استفاده شده است:

۲. مسیریابی چندگانه (Multi-Path)

هدایت مجدد بسته‌ها به مسیرهای فیزیکی مجزا در صورت بروز ازدحام در مسیر اصلی، تا ترافیک حساس از مسیرهای خلوت عبور کند.

۱. صف‌بندی و اولویت‌بندی (Linux HTB)

اختصاص صف‌های مجزا با پهنای باند مشخص به ترافیک‌ها در سطح کارت شبکه، تا ترافیک‌های سنگین باعث خرابی ترافیک‌های حساس نشوند.

۲-۴ یادگیری ماشین در مدیریت هوشمند شبکه

استفاده از قوانین ثابت و ایستای QoS پاسخگوی تغییرات ناگهانی ترافیک نیست. الگوریتم‌های یادگیری ماشین (به‌ویژه مدل‌های طبقه‌بندی / Classification) می‌توانند با دریافت پارامترهای آماری شبکه (مانند نرخ ارسال، متوسط اندازه بسته‌ها و نرخ افت)، وضعیت شبکه را به دو حالت «**عادی**» یا «**مزدحم**» طبقه‌بندی کنند.

در این سیستم، مدل ML آموزش‌دیده به عنوان مغز متفکر کنترلر عمل می‌کند؛ به این صورت که به محض تشخیص شرایط ازدحام، فرمان‌های لازم را جهت تغییر صف‌ها یا تغییر مسیر ترافیک به کنترلر صادر می‌کند.

فصل سوم: معماری سیستم و پیاده‌سازی فنی

۳-۱ نگاه کلی به معماری

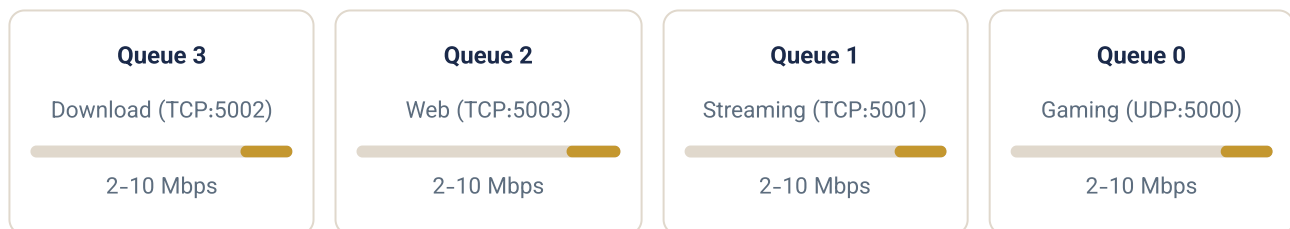
هدف از این فصل، تشریح دقیق و ساختاریافته اجزای منطقی و فیزیکی سیستم پیاده‌سازی شده است. این پروژه بر پایه سه لایه اصلی استوار است: لایه زیرساخت و شبیه‌سازی شبکه در مینی‌نت (شامل دو توپولوژی شبکه)، لایه کنترل ترافیک و صف‌بندی لینوکس (HTB)، و لایه کنترل هوشمند در محیط POX (شامل سه کنترلر با کارکردهای مجزا).

روند کلی کار به این صورت است که ابتدا شبکه در مینی‌نت راه‌اندازی شده، ترافیک توسط ابزارها تولید می‌شود، کنترلر آمار را پایش کرده و ویژگی‌های شش گانه را استخراج می‌کند. سپس این ویژگی‌ها به مدل یادگیری ماشین آموزش‌دیده (qos_mcd_model.pkl) داده می‌شوند و در نهایت بر اساس خروجی مدل، سیاست تخصیص صف یا مسیریابی پویا روی سوئیچ‌ها اعمال می‌گردد.

۳-۲ تشریح دقیق توپولوژی‌های شبکه در Mininet

توپولوژی اول (مبتنی بر صف‌بندی - QoS_ML_Topology)

این توپولوژی از یک سوئیچ مرکزی S1، یک سرور مقصد با آدرس آی‌پی 10.0.0.100/24 و ۵ میزبان (h1 تا h5) تشکیل شده است. لینک بین سوئیچ S1 و سرور به عنوان **گلوگاه شبکه** تعریف شده و پهنای باند آن ۱۰ مگابیت بر ثانیه، تاخیر آن ۲ میلی‌ثانیه و افت بسته آن صفر تنظیم شده است.



پیکربندی اولیه ۴ صف HTB روی پورت s1-eth1 (حالت متوازن)

تابع `setup_queues` بلافاصله پس از اجرای مینی‌نت، دستورات `ovs-vsctl` را اجرا کرده و روی پورت s1-eth1 کنترل ترافیک linux-htb را فعال می‌کند. در حالت اولیه، تمام ۴ صف دارای حداقل پهنای باند ۲ مگابیت بر ثانیه و حداکثر ۱۰ مگابیت بر ثانیه هستند. روی نود سرور نیز سرویس‌های iperf به صورت اتوماتیک در پس‌زمینه روی پورت‌های ۵۰۰۱، ۵۰۰۲ و ۵۰۰۳ اجرا می‌شوند.

توپولوژی دوم (مبتنی بر مسیریابی چندگانه - MultiPathQoSTopo)

این توپولوژی شامل ۸ سوئیچ (S1 تا S8)، یک سرور با آی‌پی 10.0.0.100 و ۵ میزبان (h1 تا h5) است. سه مسیر فیزیکی موازی بین مبدا و مقصد تعریف شده است:

مسیر ۱ (مسیر پیش‌فرض):



مسیر ۲ (مسیر Gaming - در صورت ازدحام):



مسیر ۳ (مسیر Streaming - در صورت ازدحام):



تمام لینک‌های بین سوئیچ‌ها دارای پهنای باند ۱۰ مگابیت بر ثانیه و تاخیر ۲ میلی‌ثانیه هستند. جهت جلوگیری از طوفان انتشار (Broadcast Loop)، دستور `net.staticArp()` اجرا شده تا جدول ARP نودها به صورت ایستا پر شود.

۳-۳ ساختار، کلاس‌ها و توابع ۳ کنترلر POX

کنترلر اول (Data Collector)

این کنترلر صرفاً در فاز راه‌اندازی اولیه پروژه استفاده شد. ساختار آن شامل یک کلاس اصلی با تایمر ۲ ثانیه‌ای است که پیام‌های `ofp_stats_request` (هم Flow Stats و هم Port Stats) را به سوئیچ ارسال می‌کند. این کنترلر میزان بایت‌ها و بسته‌ها را استخراج کرده و با محاسبه نرخ داده، فایل دیتاست اولیه را جهت آموزش مدل یادگیری ماشین تولید نمود.

کنترلر دوم (IntelligentQoSController - مبتنی بر HTB Queuing)

این کنترلر روی توپولوژی اول اجرا شده و وظیفه مدیریت صف‌ها را بر عهده دارد. توابع کلیدی:

- `handle_ConnectionUp` : نصب قوانین ثابت OpenFlow برای هدایت ترافیک هر پورت به صف مربوطه
- `handle_PortStatsReceived` : محاسبه مجموع افت بسته‌ها و ثبت میزان افت در بازه اخیر
- `handle_FlowStatsReceived` : محاسبه ۶ ویژگی اصلی و ارسال به مدل ML
- `apply_qos_policy` : افزایش پهنای باند صف گیمینگ به ۷ مگابیت و محدود کردن صف دانلود به ۵۰۰ کیلوبیت
- `remove_qos_policy` : بازگشت تمام صف‌ها به حالت متوازن قبلی

۶ ویژگی استخراج‌شده برای مدل ML:

`kbps` - نرخ ارسال داده | `kbps_diff` - تغییرات نرخ داده | `kbps_rolling_avg` - میانگین متحرک
`avg_pkt_size` - میانگین اندازه بسته | `udp_tcp_ratio` - نسبت UDP به TCP | `drop_rate` - نرخ افت بسته

کنترلر سوم (IntelligentQoSController - مبتنی بر Multi-Path Rerouting)

این کنترلر روی توپولوژی ۸ سوئیچی اجرا می‌شود. پردازش داده‌ها و فراخوانی مدل ML فقط برای آمار سوئیچ S1 (یعنی `dpid == 1`) انجام می‌شود. توابع کلیدی:

- `flood` : انتشار بسته‌های ناشناخته بدون ایجاد Broadcast Loop
- `apply_reroute_policy` : صدور پیام‌های `ofp_flow_mod` با اولویت بالا (Priority 1000) روی S1 و

S8

— `remove_reroute_policy` : حذف قوانین اولویت‌دار و بازگشت به مسیر پیش‌فرض

۳-۴ پایپ‌لاین یادگیری ماشین (ML Pipeline)

- 1 استخراج و پیش‌پردازش داده‌ها: داده‌های خام شامل ۶ ویژگی عددی از کنترلر جمع‌آوری دیتا استخراج شدند.
- 2 برچسب‌گذاری (Labeling): داده‌ها بر اساس شرایط ترافیک به دو دسته 0 (وضعیت نرمال) و 1 (وضعیت ازدحام) برچسب‌گذاری گردیدند.
- 3 آموزش و ذخیره‌سازی مدل: الگوریتم طبقه‌بندی روی داده‌ها آموزش دیده و مدل نهایی با فرمت binary در فایل `qos_mcd_model.pkl` ذخیره شد.
- 4 تزریق زنده به کنترلر: کتابخانه `joblib` در زمان مقداردهی اولیه فایل مدل را روی حافظه بارگذاری می‌کند تا پیش‌بینی به صورت لحظه‌ای اجرا شود.

۳-۵ چرخه کاری سیستم (System Workflow)

- 1 راه‌اندازی زیرساخت: شبیه‌ساز Mininet با اجرای فایل توپولوژی بالا می‌آید.
- 2 اتصال به کنترلر: کنترلر POX اجرا شده و سوئیچ‌ها از طریق پورت ۶۶۳۳ متصل می‌شوند.
- 3 تولید ترافیک: ابزارهای `iperf` و `udp_ping_client` شروع به ارسال ترافیک می‌کنند.
- 4 پایش زنده: کنترلر هر ۲ ثانیه آمار را جمع‌آوری کرده و ۶ ویژگی را به مدل ML می‌دهد.
- 5 اتخاذ تصمیم: بر اساس خروجی مدل (۰ یا ۱)، سیاست QoS اعمال یا حذف می‌شود.
- 6 بازگشت به حالت اولیه: پس از قطع ترافیک اضافه، سیاست‌ها حذف و منابع آزاد می‌شوند.

فصل چهارم: ارزیابی، سناریوهای تست و تحلیل نتایج

۴-۱ مقدمه

در این فصل، مراحل گام‌به‌گام جمع‌آوری داده، آموزش مدل یادگیری ماشین و ارزیابی عملیاتی دو رویکرد **صف‌بندی** **HTB** و **مسیریابی چندگانه (Multi-Path)** بر اساس خروجی‌های واقعی محیط شبیه‌سازی ارائه می‌شود. متدولوژی ارزیابی بر پایه مقایسه رفتاری شبکه در دو حالت «کنترلر ساده بدون QoS» و «کنترلر هوشمند اختصاصی» استوار است.

۴-۲ پیش‌نیازها و زیرساخت اجرا

تمامی فایل‌های کنترلر در مسیر `pox/ext/~` قرار دارند. جهت اجرای تست‌ها از دو محیط ترمینال همزمان استفاده شده است: ترمینال اول برای اجرای کنترلر POX و ترمینال دوم برای اجرای توپولوژی‌ها و صدور دستورات تست.

۴-۳ فرایند تولید دیتاست و آموزش مدل یادگیری ماشین

ویژگی‌های آماری شبکه در حالت‌های متوازن و مزدحم استخراج گردید. ترافیک در ۴ مرحله تزریق شد:

- 1 مرحله ۱ (ترافیک عادی - برچسب ۰): ترافیک گیمینگ ۱.۵ مگابیت و استریمینگ ۲ مگابیت
- 2 مرحله ۲ (ورود به آستانه ازدحام): اضافه شدن ترافیک دانلود ۲ مگابیت و افزایش گیمینگ
- 3 مرحله ۳ (ازدحام کامل - برچسب ۱): تزریق ترافیک سنگین ۴+۴ مگابیت گیمینگ و ۵ مگابیت وب
- 4 مرحله ۴ (خروج از ازدحام): کاهش ترافیک به حالت عادی

دستور آموزش مدل:

```
sudo python3 train_model.py
```

۴-۴ سناریوی تست اول: ارزیابی رویکرد صف‌بندی HTB

گام اول: تست پایه با کنترلر ساده

اجرای کنترلر ساده و توپولوژی:

```
./pox.py forwarding.l2_learning  
sudo python3 topology.py  
mininet> server python3 udp_echo_server.py &  
mininet> h2 python3 udp_ping_client.py
```

نتیجه: تاخیر RTT در شبکه خلوت

ms **2.48**

پس از تزریق ترافیک دانلود روی پورت ۵۰۰۲ توسط میزبان h4:

نتیجه: تاخیر RTT در حالت ازدحام (بدون QoS)

ms **25.20** (اوج: ms 54.56)

گام دوم: تست با کنترلر هوشمند اول

اجرای کنترلر هوشمند:

```
python3 pox.py qos_controller  
sudo python3 topology.py
```

به محض ورود ترافیک، مدل ML وضعیت (1) CONGESTED را تشخیص داد:

```
[WARN] Features: kbps=8240, kbps_diff=3200, avg_pkt_size=512, udp_tcp_ratio=1.8,  
drop_rate=0.012  
[ML] Prediction: 1 (CONGESTED)  
[QoS] QoS Policy Applied: Gaming priority boosted (Min 7Mbps), Bulk download  
throttled.
```

نتیجه: تاخیر RTT با کنترلر هوشمند HTB

♦ کاملاً ثابت و پایدار ms **2.28**

با QoS هوشمند (HTB + ML)

2.28 ms

تاخیر ثابت و حتی کمتر از حالت خلوت – کاهش ۹۱٪

بدون QoS (کنترلر ساده)

25.20 ms

افزایش ۱۰ برابری تاخیر به دلیل اشتراک صف و عدم اولویت‌بندی

۴-۵ سناریوی تست دوم: ارزیابی رویکرد مسیریابی چندگانه

گام اول: تست پایه با کنترلر ساده

جهت اجبار ترافیک به عبور از مسیر اول، لینک‌های جانبی خاموش شدند:

اجرای تست مسیریابی چندگانه - کنترلر ساده:

```
./pox.py forwarding.l2_learning
sudo python3 topology2.py
mininet> s1 ifconfig s1-eth3 down
mininet> s1 ifconfig s1-eth4 down
mininet> server python3 udp_echo_server.py &
mininet> h2 python3 udp_ping_client.py
```

نتیجه: تاخیر RTT در حالت خلوت (مسیر طولانی‌تر)

15.60 ms

پس از تزریق ترافیک دانلود `iperf h4` روی پورت ۵۰۰۲:

نتیجه: تاخیر RTT در حالت ازدحام (بدون Rerouting)

478.13 ms ⚠ جهش بحرانی!

گام دوم: تست با کنترلر هوشمند دوم

اجرای کنترلر چندمسیره هوشمند:

```
python3 pox.py qos_controller2
sudo python3 topology2.py
mininet> server python3 udp_echo_server.py &
mininet> h2 python3 udp_ping_client.py
```

با تزریق ترافیک دانلود، مدل ML هشدار ازدحام صادر کرد:

```
[WARN] Features extracted from S1: kbps=9800, drop_rate=0.038
[ML] Prediction: 1 (CONGESTED)
[REROUTE] Congestion predicted! Rerouting Gaming -> Path 2 (S3-S6) & Stream -> Path 3 (S4-S7)
[REROUTE] Multi-Path Active: Forward & Return paths installed on S1 and S8.
```

نتیجه: تاخیر RTT با کنترلر هوشمند چندمسیره

15.01 ms ♦ پایدار و بدون قطعی

با Rerouting هوشمند (ML)

15.01 ms

تاخیر بدون تغییر نسبت به حالت خلوت – کاهش ۹۷٪

بدون Rerouting (کنترلر ساده)

478.13 ms

انسداد کامل مسیر – تاخیر تقریباً نیم ثانیه، عملاً غیرقابل استفاده برای Gaming

۴-۶ جدول مقایسه جامع نتایج عددی ارزیابی

سناریو / نوع کنترلر	RTT حالت خلوت	RTT حالت ازدحام	وضعیت اکشن کنترلر
سناریو ۱ - کنترلر ساده	ms 2.48	ms 25.20	عدم تغییر (ترافیک در صف عمومی)
سناریو ۱ - کنترلر HTB	ms 2.45	ms 2.28	ارتقای پهنای باند Queue 0 به 7Mbps
سناریو ۲ - کنترلر ساده	ms 15.60	ms 478.13	انسداد مسیر و گلوگاه شدن لینک
سناریو ۲ - کنترلر چندمسیره	ms 15.20	ms 15.01	هدایت گیمینگ به پورت ۳ و استریم به پورت ۴

نکته کلیدی: در هر دو سناریو، کنترلر هوشمند توانسته تاخیر ترافیک حساس Gaming را در شرایط ازدحام شدید، دقیقاً در همان سطح حالت خلوت نگه دارد. این نشان‌دهنده عملکرد موفق مدل ML در تشخیص

سریع و اعمال سیاست صحیح است.

فصل پنجم: نتیجه‌گیری و پیشنهادات

۵-۱ نتیجه‌گیری

در این تحقیق، یک سیستم مدیریت خودکار و هوشمند کیفیت خدمت (QoS) برای شبکه‌های مبتنی بر نرم‌افزار (SDN) با استفاده از تکنیک‌های یادگیری ماشین طراحی و پیاده‌سازی گردید. با هدف ارتقای پارامترهای کارایی برای برنامه‌های کاربردی حساس به تاخیر (مانند بازی‌های آنلاین و پخش زنده ویدیو)، دو رویکرد مجزای «صف‌بندی پویا (Linux HTB)» و «مسیریابی چندگانه (Multi-Path Rerouting)» در بستر کنترلر POX و شبیه‌ساز Mininet توسعه یافت.

نتایج کلیدی و دستاوردهای حاصل از ارزیابی عملیاتی سیستم به شرح زیر است:

۱ **تشخیص دقیق و زنده شرایط ازدحام:** مدل یادگیری ماشین آموزش‌دیده با استخراج زنده ویژگی‌های ۶ گانه شبکه، قادر است در کمتر از ۴ ثانیه بروز ازدحام را تشخیص داده و سیگنال تغییر سیاست را به کنترلر صادر کند.

۲ **پایداری تاخیر در رویکرد صف‌بندی HTB:** در توپولوژی تک‌سوئیچی، با اعمال سیاست هوشمند اولویت‌بندی صف‌ها، تاخیر RTT ترافیک حساس Gaming در شرایط ازدحام شدید از متوسط **25.20 میلی‌ثانیه** به **2.28 میلی‌ثانیه** تقلیل یافت.

۳ **توزیع بار بهینه در رویکرد مسیریابی چندگانه:** در توپولوژی ۸ سوئیچی، تزریق ترافیک متجاوز باعث جهش تاخیر مسیر اصلی به **478.13 میلی‌ثانیه** گردید. با مداخله کنترلر هوشمند و هدایت مجدد ترافیک، تاخیر روی **15.01 میلی‌ثانیه** تثبیت شد.

۴ **عملکرد متقارن و بدون قطعی:** با ثبت قوانین بازگشتی متناظر روی سوئیچ مقصد (S8)، هر دو رویکرد توانستند ارتباطات دوطرفه UDP و TCP را بدون حتی یک قطع اتصال هدایت کنند.

در مجموع، نتایج آزمایش‌ها اثبات می‌کند که تلفیق یادگیری ماشین با پایش زنده OpenFlow، ناکارآمدی‌های مدیریت ایستای شبکه را برطرف کرده و راهکاری انعطاف‌پذیر برای تضمین QoS در شبکه‌های نسل جدید ارائه می‌دهد.

۵-۲ پیشنهادات برای تحقیقات آتی

برای توسعه، بهبود و ارتقای کارایی این پروژه در تحقیقات آینده، راهکارهای زیر پیشنهاد می‌شود:

- 1 به‌کارگیری مدل‌های یادگیری عمیق و تقویت‌یابی (DRL): استفاده از الگوریتم‌های «یادگیری تقویت‌یابی عمیق» مانند DQN یا PPO جهت اخذ تصمیمات مسیریابی پویاتر بدون نیاز به برچسب‌گذاری دستی داده‌ها.
- 2 پشتیبانی از پروتکل OpenFlow 1.3 و گروه‌ها (Group Tables): ارتقای نسخه پروتکل OpenFlow جهت بهره‌گیری از جدول گروه‌ها برای اجرای تکنیک‌های اشتراک بار بر اساس وزن لینک‌ها.
- 3 ارزیابی در توپولوژی‌های واقع‌گرایانه‌تر: تست سیستم هوشمند روی توپولوژی‌های پیچیده و مقیاس‌بزرگ منطقه‌ای (WAN) مانند GEANT یا NSFNET.
- 4 ترکیب همزمان دو رویکرد (Hybrid QoS): طراحی یک کنترلر جامع که در صورت تکمیل ظرفیت صف‌های HTB، به صورت خودکار مکانیزم تغییر مسیر را نیز فعال کند.
- 5 ارزیابی امنیت و مقابله با حملات DoS/DDoS: ارتقای مدل ML جهت تفکیک ترافیک ازدحام واقعی از حملات انکاری خدمت و اعمال سیاست‌های امنیتی و QoS به صورت همزمان.

دانشگاه صنعتی اصفهان | دانشکده مهندسی کامپیوتر | تابستان ۱۴۰۴

تهیه‌شده توسط شایان رواخواه — زیر نظر دکتر مسعود هاشمی